第4章 处理器调度与死锁

4.1 处理器调度分级模型

4.2 调度算法

4.3 实时调度

4.4 死锁

4.1 处理器调度分级模型

本章将以CPU 管理为核心，讨论管理、控制用户进程执行的方法。

主要包括：

- 计算机系统中存在哪些调度？

  处理机调度分级模型

- 处理机分配给谁？

  调度策略

- 怎样评价调度策略的好坏？

  调度策略评价指标

- 调度怎么实现？

  调度时机和调度的具体流程

4.1.1 作业和作业的状态转换

- 作业基本概念
  - 多道批处理系统中，作业（JCB）可被看作是用户向计算机提交任务的任务实体，例如一次计算、一个控制过程等。
  - 用户提交的作业通过相应的输入设备输入到磁盘存储器，并保存到一个后备作业队列中，再由作业调度程序将其由外存调入内存。

4.1.1 作业和作业的状态转换

- 作业控制块（JCB）

作业名
作业类型
资源要求
资源使用情况
优先级(数)
当前状态
其他

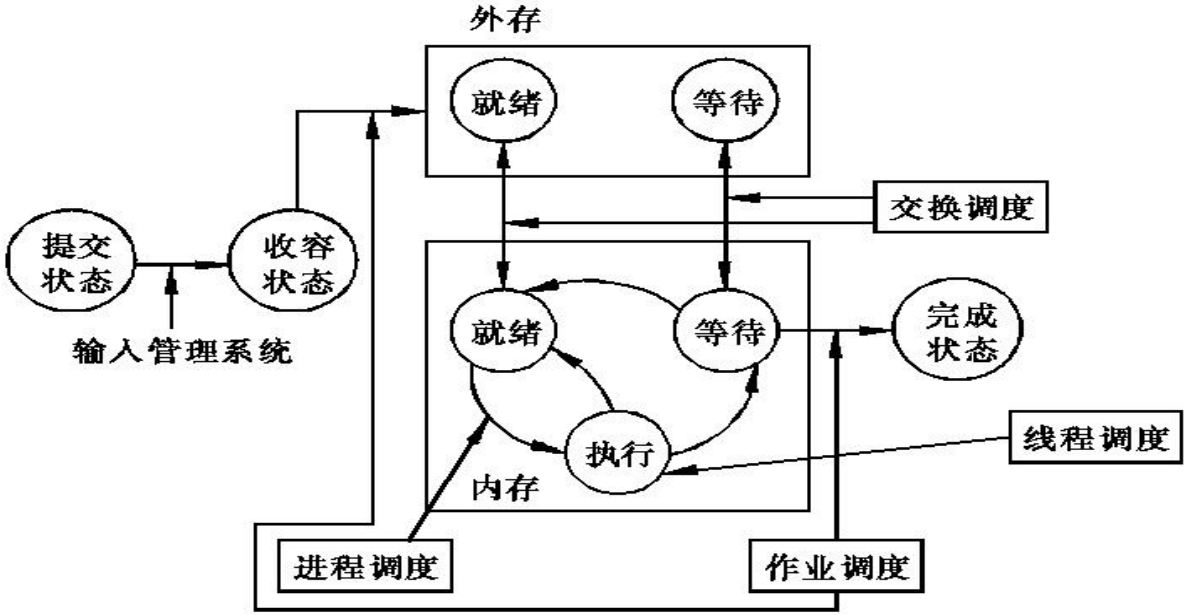
图4.1 作业控制块JCB

4.1.1 作业和作业的状态转换

- 作业与进程
  - 进程是计算机为了完成用户任务实体而设置的执行实体，是系统分配资源的基本单位。
  - 一个作业是由一个以上的多个进程组成的。
  - 作业分解为进程：
    - 根进程->子进程

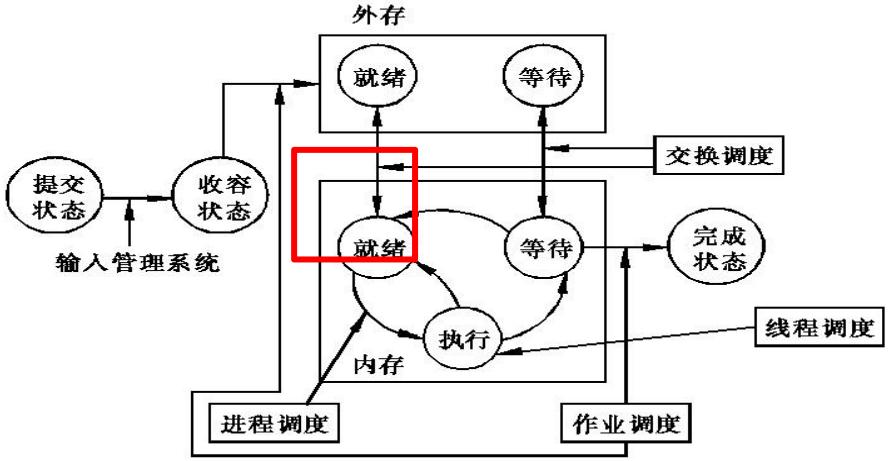
4.1.1 作业和作业的状态转换

- 作业的状态
  - 一个作业从提交给计算机系统到执行结束退出系统，一般要经历提交、收容、执行和完成等4个状态。



4.1.1 作业和作业的状态转换

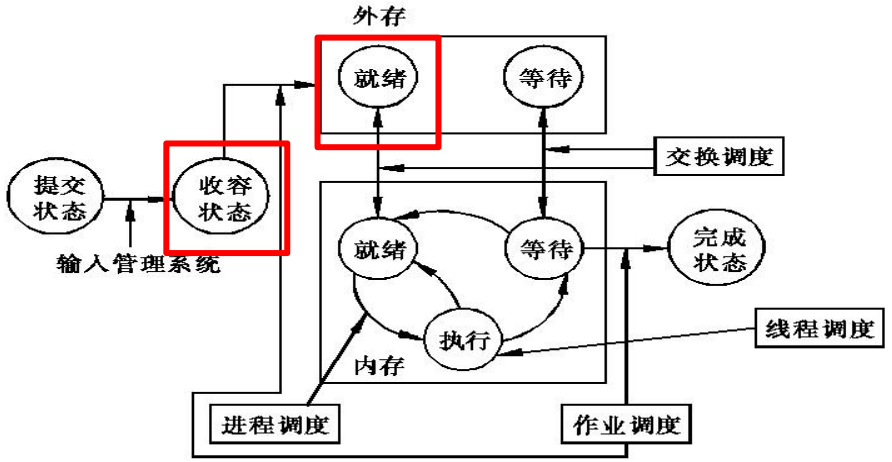
- 作业的状态



  - **提交状态**：一个作业**在其处于从输入设备进入外部存储设备的过程**称为提交状态。处于提交状态的作业，因其信息尚未全部进入系统，所以不能被调度程序选取。

4.1.1 作业和作业的状态转换

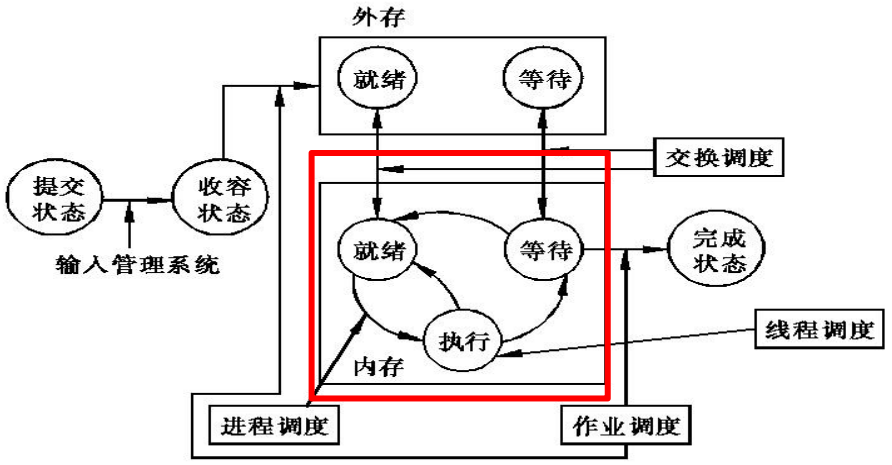
- 作业的状态



  - **收容状态**：也称为后备状态。输入管理系统不断地将作业输入到外存中对应部分。若**一个作业的全部信息已全部被输入进输入井，那么，在它还未被调度去执行之前**，该作业处于收容状态。

4.1.1 作业和作业的状态转换

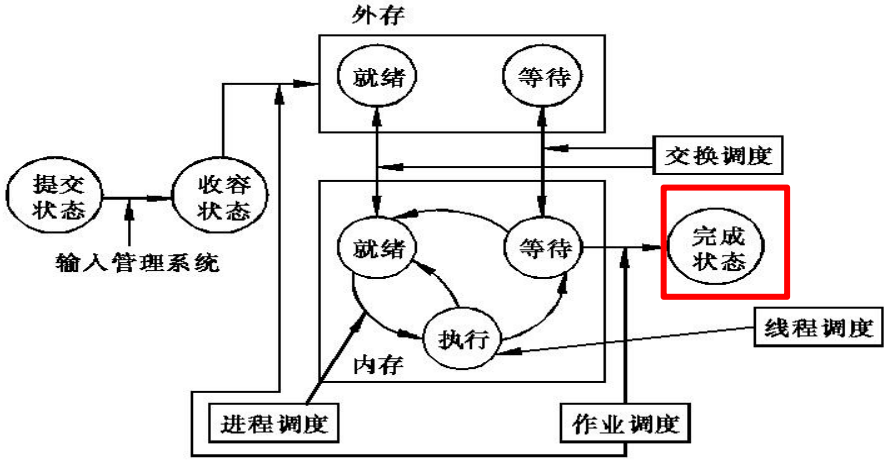
- 作业的状态



  - **执行状态**：作业调度程序从后备作业中选取若干个作业**到内存投入运行。它为被选中作业建立进程并分配必要的资源**，这时，这些被选中的作业处于执行状态。

4.1.1 作业和作业的状态转换

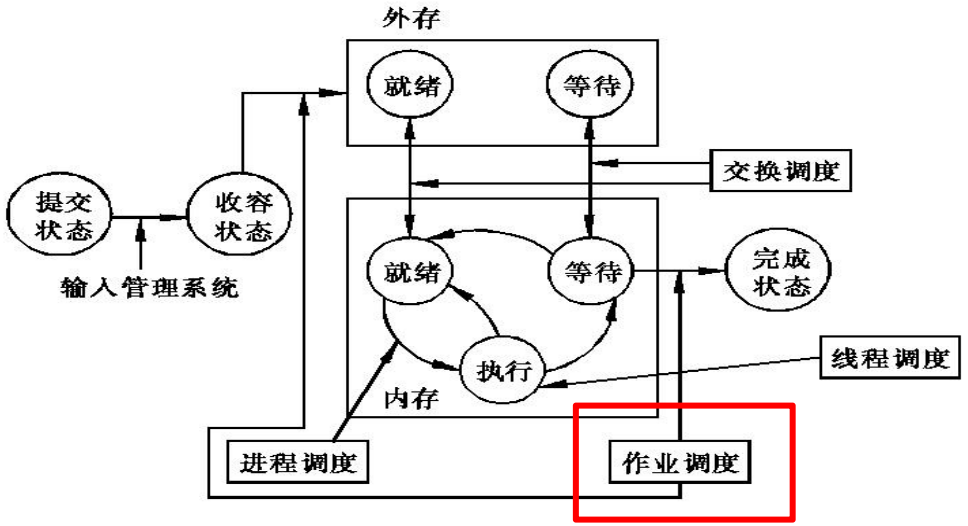
- 作业的状态



  - 完成状态：当作业运行完毕，但它所占用的资源尚未全部被系统回收时，该作业处于完成状态。在这种状态下，系统需做诸如打印结果、回收资源等类的善后处理工作。

4.1.2 处理机调度的层次

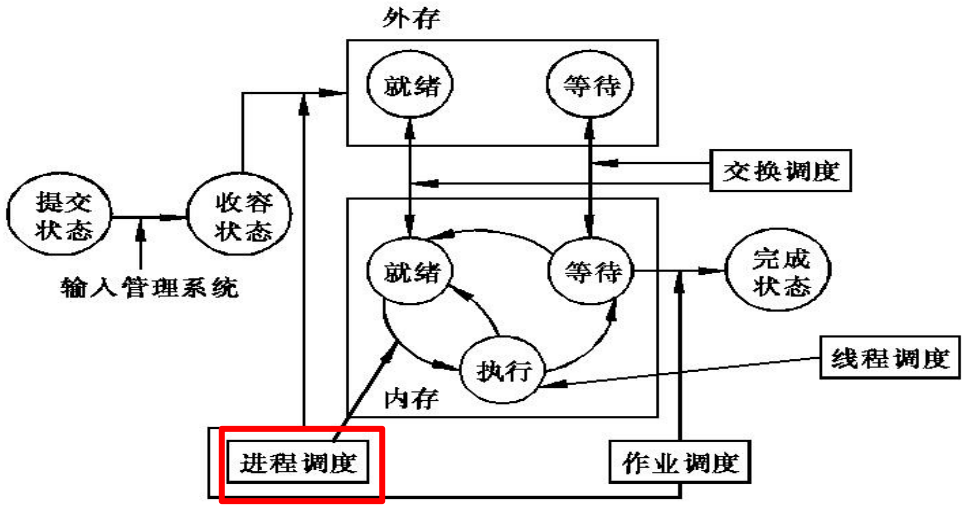
- 处理机调度的层次



- 作业调度：又称宏观调度，或高级调度、长程调度。主要任务是：按一定的原则对外存输入井上的后备作业进行选择，给选出的作业分配内存、输入输出设备等必要的资源，并建立相应的进程，使该作业的进程获得竞争处理机的权利。当该作业执行完毕时，还负责回收系统资源。

4.1.2 处理机调度的层次

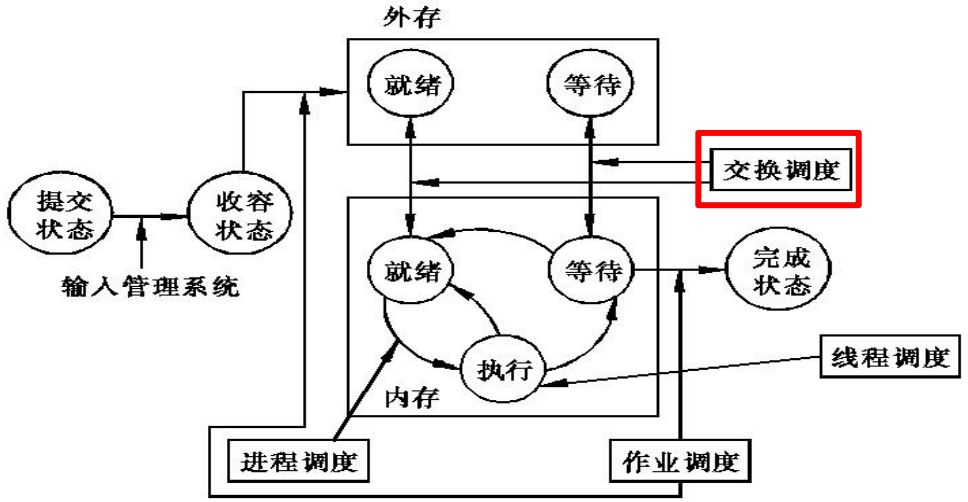
- 处理机调度的层



- **进程调度**：又称**低级调度**或**短程调度**。其主要任务是按照某种策略和方法**选取一个处于就绪状态的进程占用处理机**。在确定了占用处理机的进程后，系统必须进行进程上下文切换以建立与占用处理机进程相适应的执行环境。

4.1.2 处理机调度的层次

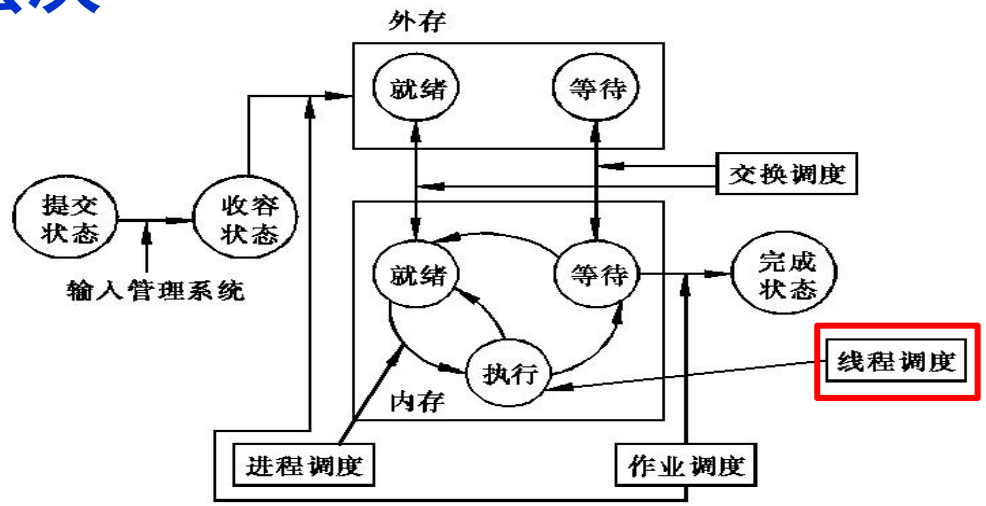
- 处理机调度的层次



- **交换调度**：又称中级调度。其主要任务是按照给定的原则和策略，将处于**外存交换区**中的就绪状态或就绪等待状态的进程**调入内存**，或把处于内存就绪状态或内存等待状态的进程**交换到外存**交换区。交换调度主要涉及到内存管理与扩充。

4.1.2 处理机调度的层次

- 处理机调度的层次



- **线程调度**：在有线程的操作系统中，线程调度是指按照特定策略为某个线程**分配CPU的使用权**。在确定了占用处理机的线程后，必须进行线程上下文切换以建立与占用处理机线程相适应的执行环境。

4.1.2 处理机调度的层次

- 处理机调度的层次
  - 多道批处理系统：作业调度和进程调度
  - 分时系统和实时系统中：进程调度、交换调度和线程调度

4.1.3 调度过程与性能衡量

- 处理机调度的目标
  - 共同目标
    - 资源利用率：系统中的处理机和其他所有资源都尽可能的保持忙碌状态。
    - 公平性：各个进程都获得合理的CPU时间，避免进程“饿死”。
    - 平衡性：系统使用尽量平衡，避免一个资源（如CPU)忙个不停，一个资源（如I/O)无所事事。
    - 策略强制执行：特殊情况的强制策略/安全策略。

4.1.3 调度过程与性能衡量

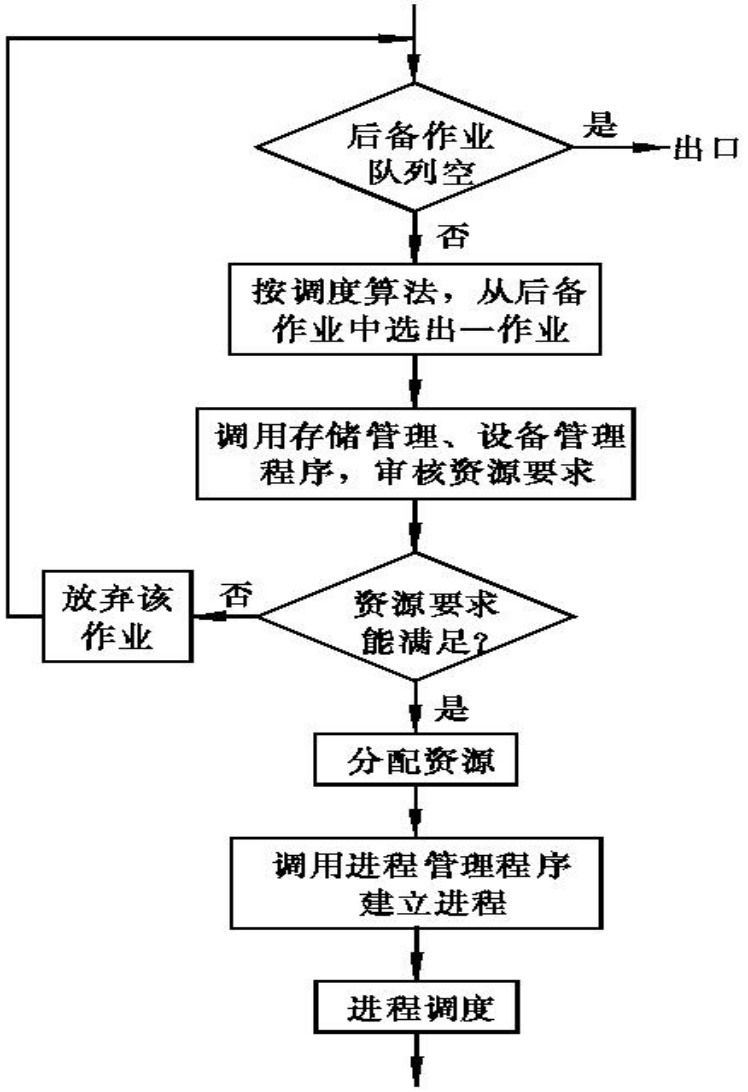
- 处理机调度的目标
  - 批处理系统
    - 周转时间：是指将一个作业提交给计算机系统后到该作业的结果返回给用户所需要的时间。
    - 公平性：各个进程都获得合理的CPU时间，避免进程“饿死”。
    - 平衡性：系统使用尽量平衡，避免一个资源（如CPU)忙个不停，一个资源（如I/O)无所事事。
    - 策略强制执行：特殊情况的强制策略/安全策略。

4.1.3 调度过程与性能衡量

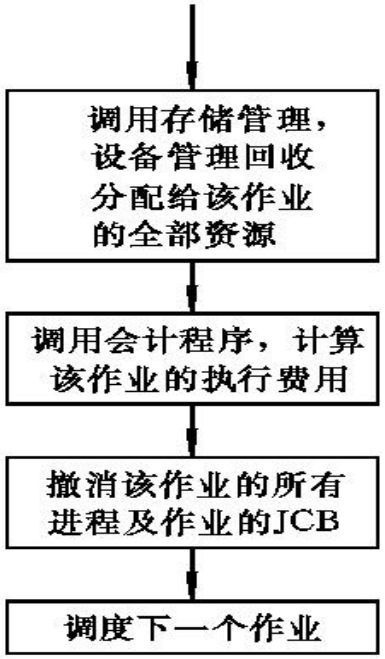
- 处理器调度的目标
  - 分时系统
    - 响应时间：则是指从用户向计算机发出一个命令到计算机把相应的执行结果返回给用户所需要的时间。
    - 均衡性：系统响应时间的快慢应与用户请求任务的复杂性相适应。
  - 实时系统
    - 截止时间保证：保证实时任务队截止时间的要求。
    - 可预测性：

4.1.4 作业调度过程与性能衡量

- 作业调度



（a）作业从后备状态到执行状态



（b）作业从执行状态到完成状态

图4.2 作业调度过程

4.1.4 作业调度过程与性能衡量

- 作业调度目标与性能衡量
  - 周转时间：作业提交到作业完成经历的时间

$$T_i = T_{ei} - T_{si}$$

其中T_{ei}为作业i的完成时间，作业T_{si}为作业的提交时间

  - 平均周转时间：设被测定作业流含有n（n>=1）个作业来说，其平均周转时间为：

$$T = \frac{1}{n}\sum_{i=1}^{n} T_i$$

4.1.4 作业调度过程与性能衡量

- 作业调度目标与性能衡量
  - 带权周转时间：是作业周转时间与作业执行时间的比。

$$W_i=T_i/T_{ri}$$

其中T_i为作业i的周转时间，作业T_{ri}为作业的执行时间

  - 平均带权周转时间：

$$w=\frac{1}{n}\sum_{i=1}^{n}w_i$$

4.3.2 进程调度过程与性能衡量

- 进程上下文（Process Context）
  - 进程的上下文是指进程在执行过程中所依赖的环境和状态信息的集合，它描述了一个进程在执行过程中，某一时刻的完整状态信息.
  - 这些信息是操作系统进行进程调度、切换和恢复时必需的数据，确保进程被中断后能够从“断点”继续正确执行。
  - 简单来说，进程上下文是进程运行时的“快照”，保存了进程在 CPU 和内存中的关键状态。

4.1.4 进程调度过程与性能衡量

- 进程上下文
  - 进程上下文切换时，需要保存的上下文信息通常包括：
    - **用户级上下文**：包括用户空间的程序计数器、寄存器（如通用寄存器、浮点寄存器等）、用户栈指针等。这些信息反映了进程在用户空间中的执行状态。
    - **系统级上下文**：包括内核态的寄存器、内核栈指针、进程状态、程序入口地址、优先级等。这些信息反映了进程在内核空间中的执行状态。
    - **其他信息**：除了上述信息外，还可能包括页表、打开文件表等其他与进程相关的状态信息。

4.1.5 进程调度过程与性能衡量

- **进程调度过程**
  - **决定是否做上下文切换以及是否允许做上下文切换：** 包括对进程调度原因的检查分析，以及当前执行进程的资格和CPU执行方式的检查等。如：当前进程处于不可中断的睡眠状态；系统资源不足；进程调度器锁定；中断处理程序执行中等。
  - **进程上下文切换**
    - **保存当前进程的上下文信息**：操作系统将当前进程的上下文信息保存到PCB或与之相关的系统内核结构中。
    - **选择下一个要运行的进程：** 调度器根据调度策略和算法，从就绪队列中选择下一个要运行的进程。
    - **加载下一个进程的上下文信息：** 操作系统从PCB或与之相关的系统内核结构中检索下一个进程的上下文信息，并将其加载到CPU的寄存器中。
    - **切换到新进程：** 操作系统将控制权转交给新进程，新进程从之前保存的上下文信息中恢复执行。

4.2.5 进程调度过程与性能衡量

- 进程调度过程

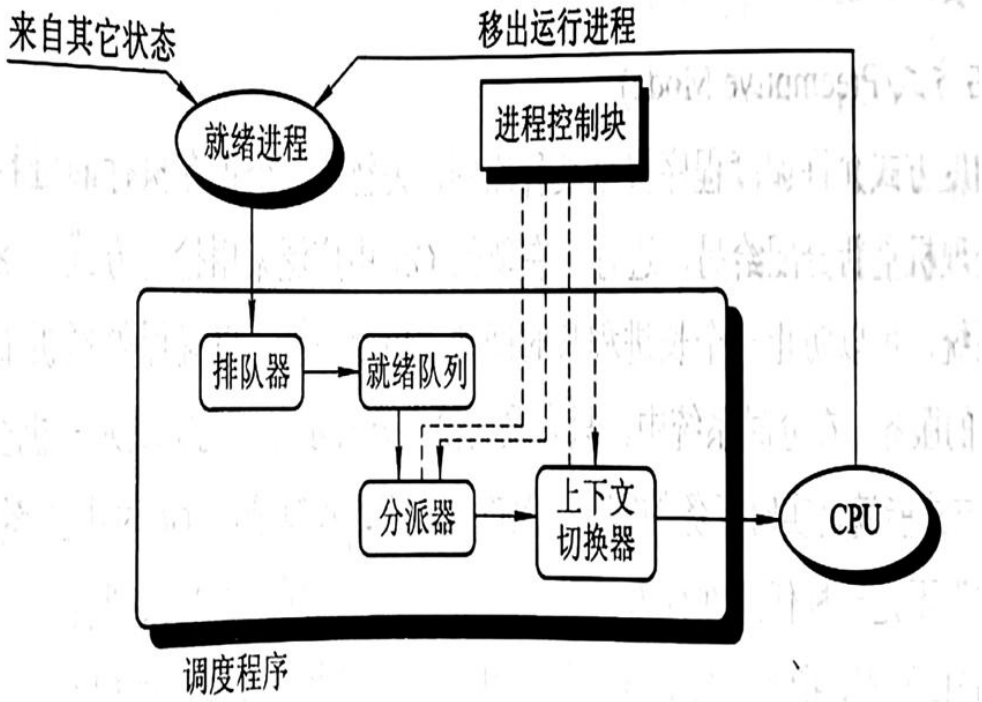


图4.3 进程调度过程

4.1.5 进程调度过程与性能衡量

- 进程调度的原因
  - 当进程执行中放弃了处理器时：
    - 主动放弃：exit，调用wait等进入阻塞状态时等。
    - 被动放弃：P原语，I/O阻塞等。
    - 时间片用完。
  - 当进程由系统态转入用户态时：
    - 当前进程系统调用执行结束后返回用户态时。
    - 当前进程在完成中断和陷阱处理后返回用户态时，

      若当前进程的优先级已低于其他就绪状态进程，内核可能直接切换进程；或设置调度标志位，在后续返回用户态时触发调度。

4.1.5 进程调度过程与性能衡量

- **进程调度方式**
  - **可剥夺方式（抢占式）：** **即就绪队列中一旦有优先级高于当前执行进程优先级的进程存在时，便立即发生进程调度，转让处理机。**
  - **非剥夺方式（非抢占式）**：即使在就绪队列存在有优先级高于当前执行进程时，当前进程仍将继续占有处理机，直到该进程自己因调用原语操作或等待I/O而进入阻塞、睡眠状态，或时间片用完时才重新发生调度让出处理机。

4.1.5 进程调度过程与性能衡量

- 进程调度性能评价
  - 进程调度性能的衡量方法可分为定性和定量两种
  - 在定性衡量方面，首先是调度的可靠性。另外，简洁性也是衡量进程调度的一个重要指标。
  - 进程调度的定量评价包括CPU的利用率评价、进程在就绪队列中的等待时间与执行时间之比等。
  - 一般情况下，大多利用模拟或测试系统响应时间的方法来评价进程调度的性能。

4.2 调度算法

- 先来先服务调度算法
- 短作业(进程)优先调度算法
- 高优先权优先调度算法
- 高响应比优先调度算法
- 基于时间片的轮转调度算法

4.2 调度算法

- **先来先服务调度算法**
  - FCFS调度算法是一种最简单的调度算法。既可用于作业调度，也可用于进程调度。
  - 用于作业调度中：
    - 从后备队列作业中，选择一个或几个最先进入该队列的作业，将它们调入内存，为它们分配资源、创建进程，然后放入进程就绪队列。
  - 用于进程调度时：
    - 从进程就绪队列中，选择一个（单CPU下）最先进入该队列的进程，为之分配处理机，使之投入运行。该进程一直运行到完成或发生某事件而阻塞后，才放弃处理机。（非抢占式）

4.2 调度算法

- 例：设在单道系统中用FCFS算法调度如下作业，请完成下表。

进程名	A	B	C	D	E	平　均
到达时间	9:00	9:10	9:30	10:00	10:15	
服务时间	30分钟	1小时	10分钟	50分钟	20分钟	
完成时间	9:30	10:30	10:40	11:30	11:50	
周转时间	30分钟	80分钟	70分钟	90分钟	95分钟	73分钟
带权周转时间	1	1.33	7	1.8	4.75	3.176

- FCFS优势：管理简单；

  劣势：利于长作业（进程），不利于短作业（进程）。

4.2 调度算法

- **短作业/进程优先调度算法**
  - **短作业优先（SJF）：**
    - 从后备队列中选择一个或几个估计运行时间最短的作业，将它调入内存运行。
  - **短进程优先：**
    - 从就绪队列中选择一个估计运行时间（单CPU下）最短的进程，将处理机分配给它，使它立即执行并一直到完成，或发生某事件而被阻塞放弃处理机时，再重新调度。（非抢占式）

4.2 调度算法

- 例：设在单道系统中用SJF算法调度如下作业，请完成下表。

进程名	A	B	C	D	E	平　均
到达时间	9:00	9:10	9:30	10:00	10:15	
服务时间	30分钟	1小时	10分钟	50分钟	20分钟	
完成时间	9:30	10:40	9:40	11:50	11:00	
周转时间	30分钟	90分钟	10分钟	110分钟	45分钟	57分钟
带权周转时间	1	1.5	1	2.2	2.25	1.59

4.2 调度算法

- **短作业/进程优先调度算法**
  - **优势：**
    - 当多个作业同时到达时，SJF算法可使平均周转时间最短。
  - **劣势：**
    - 该算法对长作业不利——长作业可能长期不被调度，“饿死”。
    - 未考虑作业的紧迫性，不能保证紧迫作业（进程）会被及时调度。
    - 由于作业（进程）的长短只是根据用户所提供的估计时间而定的，致使该算法不一定能真正做到短作业优先调度。

4.4 调度算法

- **高优先权优先调度算法**

  为了照顾紧迫型作业，使之在进入系统后便获得优先处理。

  - **优先权作业调度**
    - 系统从后备中选择一个或几个优先权最高的作业，将它调入内存运行。
  - **优先权进程调度**
    - 系统将处理机分配给就绪队列中优先权最高的进程。

4.2 调度算法

- 高优先权优先调度算法
  - 非抢占式优先权算法
    - 系统一旦把处理机分配给就绪队列中优先权最高的进程后，该进程便一直执行下去，直到完成，或因发生某事件使该进程放弃处理机时，系统方可再将处理机重新分配给另一个优先权最高的进程。
  - 抢占式优先权算法
    - 系统把处理机分配给就绪队列中优先权最高的进程，使之执行，但在其执行期间，只要出现了另一个优先权更高的进程，系统就立即停止当前进程的执行，重新将处理机分配给新的优先权最高的进程(常用于实时系统中以及对实时性能要求较高的批处理系统和分时系统中)

4.2 调度算法

- 高优先权优先调度算法
  - 优先权类型
    - **静态优先权**：它是在创建进程时确定就进程优先权，且在进程整个运行期间优先权保持不变。
      - 优点：简单易行，系统开销小。
      - 缺点：优先权低的作业或进程可能长期得不到调度
      - 确定优先权的依据：进程类型；进程资源需求；用户进程紧迫程度、用户支付情况等
    - **动态优先权**：在创建进程时所赋予的优先权，可以随进程得推进，或随其等待时间的增加而改变的，以便获得更好的调度性。
    - 例：UNIX System V中：

      p_pri=p_cpu/2+PUSER+p_nice+NZERO

4.2 调度算法

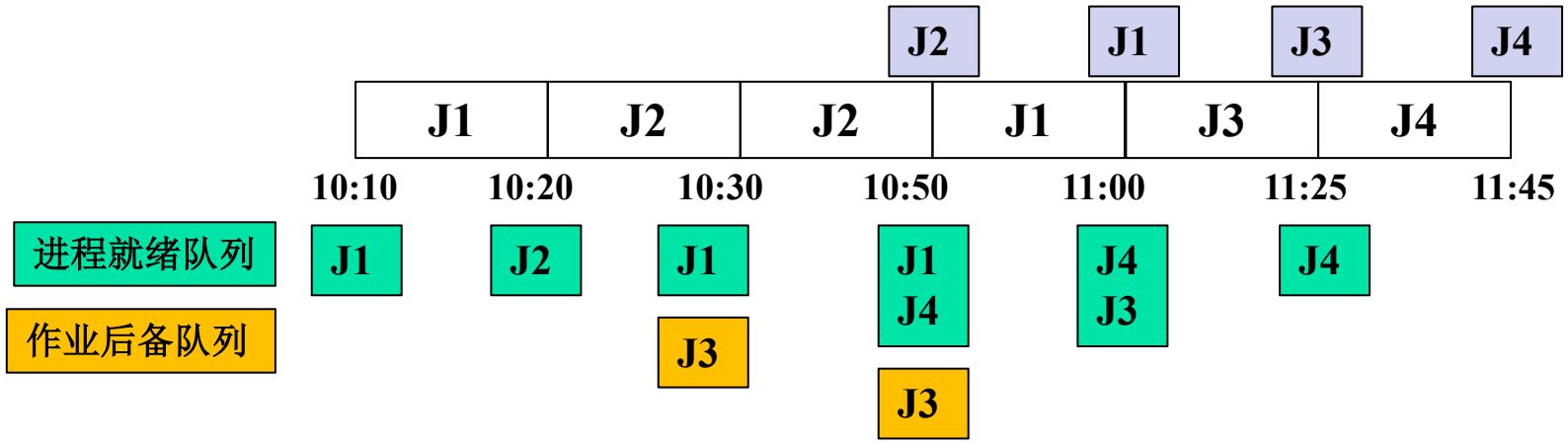
- 高优先权优先调度算法
  - 优先权类型
    - 静态优先权：它是在创建进程时确定就进程优先权，且在进程整个运行期间优先权保持不变。
      - 优点：简单易行，系统开销小。
      - 缺点：优先权低的作业或进程可能长期得不到调度
      - 确定优先权的依据：进程类型；进程资源需求；用户进程紧迫程度、用户支付情况等
    - 动态优先权：在创建进程时所赋予的优先权，可以随进程得推进，或随其等待时间的增加而改变的，以便获得更好的调度性。
    - 例：UNIX System V中：

      p_pri=p_cpu/2+PUSER+p_nice+NZERO

4.2 调度算法

- 例1：设有一个最多可有两道作业同时装入内存执行的批处理系统，作业调度采用最短作业优先调度算法，进程调度采用抢占式静态优先权调度算法，今有如下纯计算型作业序列（表中所列进程优先数中，数值越小优先权越高）：

作业名	到达时间	估计运行时间	进程优先数
J1	10:10	20分钟	5
J2	10:20	30分钟	3
J3	10:30	25分钟	4
J4	10:50	20分钟	6

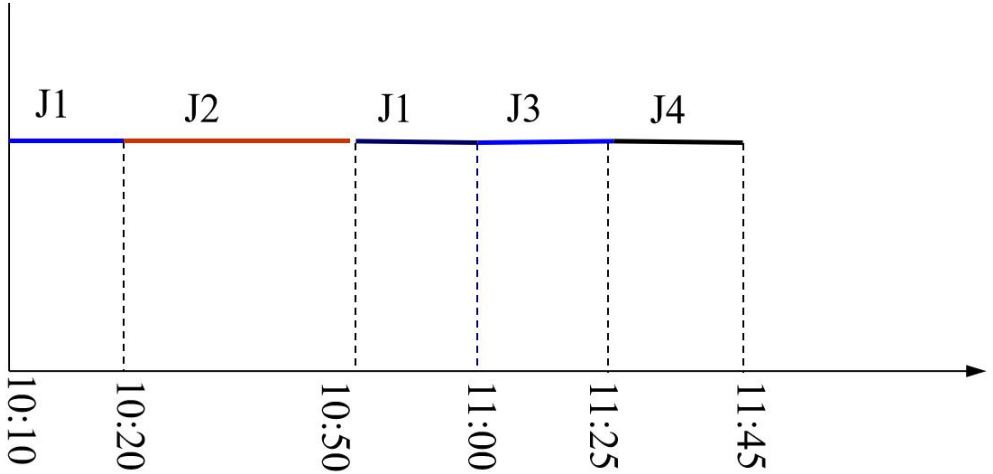


4.2 调度算法

- 例1：分析详细过程如下：
  - 10：10　J1到达，作业调度程序将J1调入内存执行
  - 10：20　J2到达，作业调度程序将J2调入内存，J2优先级高于J1，抢占J1，进程调度程序调度J2运行，J1转入进程就绪队列，J1已运行10分钟
  - 10：30　J3到达，已有J1和J2在内存，J3不调入内存
  - 10：50　J2结束，J4到达，J4作业时间短于J3，作业调度程序将J4调入内存。J1优先级高于J4，进程调度程序调度J1运行
  - 11：00　J1结束，作业调度程序调度J3到内存。J3优先级高于J4，进程调度程序调度J3运行
  - 11：25　J3结束，无作业调入内存。进程调度程度调度J4运行
  - 11：45　J4结束

4.2 调度算法

- 例1：



作业名	到达时间	调入时间	结束时间	周转时间
J1	10:10	10:10	11:00	50分钟
J2	10:20	10:20	10:50	30分钟
J3	10:30	11:00	11:25	55分钟
J4	10:50	10:50	11:45	55分钟

- 平均周转时间=(50+30+55+55)/4=47.5(分钟)

4.2 调度算法

- 高响应比优先调度算法
  - 响应比=(等待时间+要求服务时间)/要求服务时间
  - 高响应比作业优先调度
    - 每次要进行作业调度时，系统首先计算后备队列中各作业的响应比，然后选择一个或若干个响应比最高的作业调入内存执行
  - 高响应比进程优先调度
    - 每次要进行进程调度时，系统首先计算就绪队列中各进程的响应比，然后选择一个响应比最高的作业调入内存执行

4.2 调度算法

- 高响应比优先调度算法
  - 响应比=(等待时间+要求服务时间)/要求服务时间
  - 思考：
    - 服务时间相同，等待时间越长，则优先权越____?(高、低）;等待时间相同，服务时间越短，则优先权越_____?（高、低）长作业等待时间增长，则优先权越来_____?（高、低）
  - 优势：该算法综合了FCFS和SJF算法的优点——既考虑公平性，又考虑平均周转时间：
  - 劣势：增加了系统开销-计算响应比

4.4 调度算法

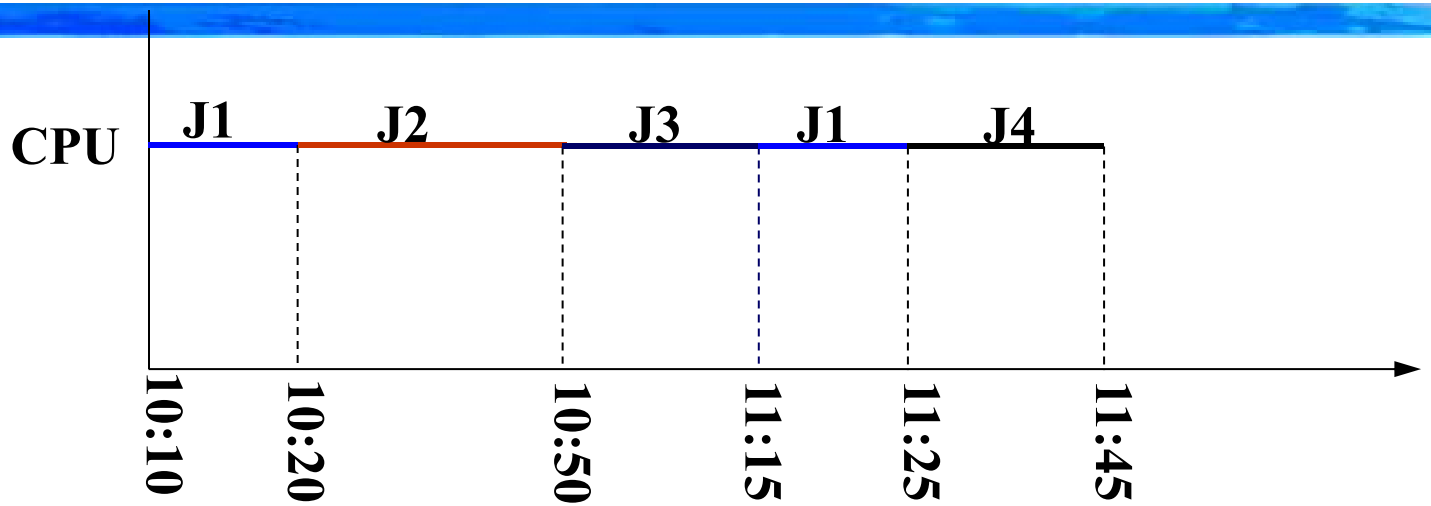
- 例2：设有一个最多可有两道作业同时装入内存执行的批处理系统，作业调度采用高响应比优先调度算法，进程调度采用抢占式静态优先权调度算法，今有如下纯计算型作业序列（假设表中所列进程优先数中，数值越小优先权越高）：

作业名	到达时间	估计运行时间	进程优先数
J1	10:10	20分钟	5
J2	10:20	30分钟	3
J3	10:30	25分钟	4
J4	10:50	20分钟	6

- (1) 列出所有作业进入内存时间及结束时间。
  (2) 计算平均周转时间。

4.2 调度算法

- 例2：



作业名	到达时间	调入时间	结束时间	周转时间
J1	10:10	10:10	11:25	75分钟
J2	10:20	10:20	10:50	30分钟
J3	10:30	10:50	11:15	45分钟
J4	10:50	11:15	11:45	55分钟

- 平均周转时间=(75+30+45+55)/4=51.25(分钟)

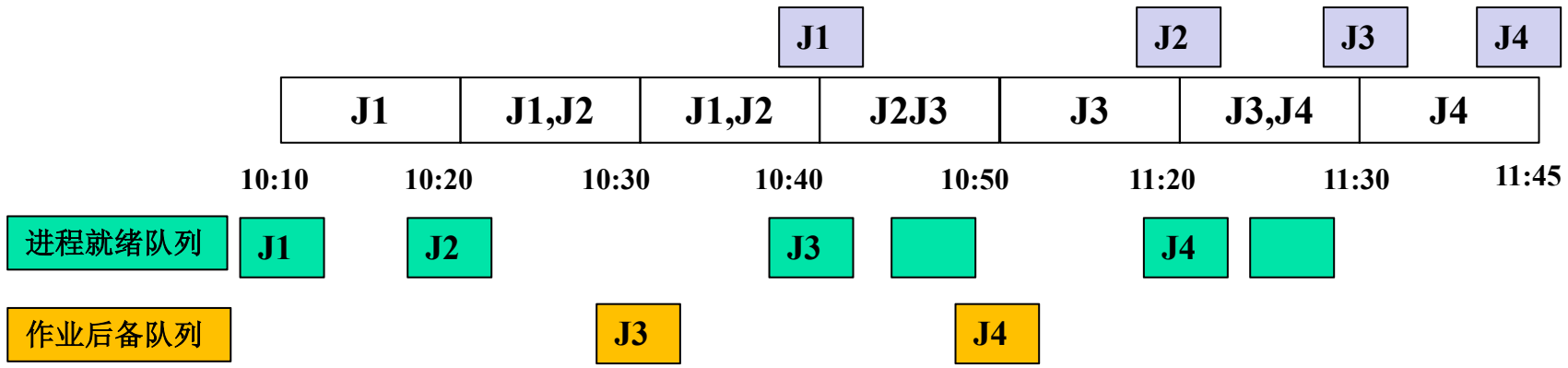
4.2 调度算法

- 基于时间片的轮转调度算法
  - 系统把就绪队列中的所有进程，按先来先服务的原则，排成一个队列；
  - 每次调度时，把CPU分配给队首进程，并让它执行一个时间片
  - 每当执行的时间片用完，调度程序便停止该进程的执行，将其送入就绪队列尾部；然后进行下一次进程调度。

4.2 调度算法

- 例3：设有一个最多可有两道作业同时装入内存执行的批处理系统，作业调度采用高响应比优先调度算法，进程调度采用时间片轮转调度算法(假设时间片为100ms)，今有如下纯计算型作业序列：

作业名	到达时间	估计运行时间	进程优先数
J1	10:10	20分钟	
J2	10:20	30分钟	
J3	10:30	25分钟	
J4	10:50	20分钟	



4.2 调度算法

- 例3：分析详细过程如下：
  - 10：10　J1到达，调入内存执行
  - 10：20　J2到达，调入内存与J1一起均分CPU运行。J1已运行10分钟，还需与J2一起运行20分钟
  - 10：30　J3到达，不调入内存
  - 10：40　J1结束，J3调入内存与J2一起均分CPU运行。J2已运行10分钟，还需与J3一起运行40分钟
  - 10：50　J4到达，不调入内存
  - 11：20　J2结束，J4调入内存与J3一起均分CPU运行。J3已运行20分钟，还需与J4一起运行10分钟
  - 11：30　J3结束，J4还需单独运行15分钟
  - 11：45　J4结束

4.2 调度算法

- 例3：
- (1) 各作业进入内存时间及结束时间如下表所示。

作业名	到达时间	调入时间	结束时间	周转时间
J1	10:10	10:10	10:40	30分钟
J2	10:20	10:20	11:20	60分钟
J3	10:30	10:40	11:30	60分钟
J4	10:50	11:20	11:45	55分钟

- 平均周转时间=(30+60+60+55)/4=51.25(分钟)

4.2 调度算法

- 多级反馈队列调度算法
  - 算法思想：
    - 就绪队列1-N：优先级递减，时间片递增
    - 一个新进程进来首先放入就绪队列1，如果在该时间片内完成，则完成。否则进程转入下一个就绪队列。在下一个就绪队列的时间片若还不能完成，转入下一个就绪队列。依次进行，直至转入到最后一个就绪队列n。
    - 同一队列采用FCFS调度算法，最后一个队列采用RR算法
    - 每次调度时，只有就绪队列1-n-1中没有就绪进程时，才从就绪队列n中选择进程调度。
    - 当来自就绪队列i中的进程执行时，如果就绪队列1-i-1有新进程到达，则抢占当前正执行的进程。
  - 优势：不必事先知道各进程所需的执行时间，而且还可以满足各种类型进程的需要，如终端型作业用户，短批处理作业用户，长批处理作业用户。

4.4 调度算法

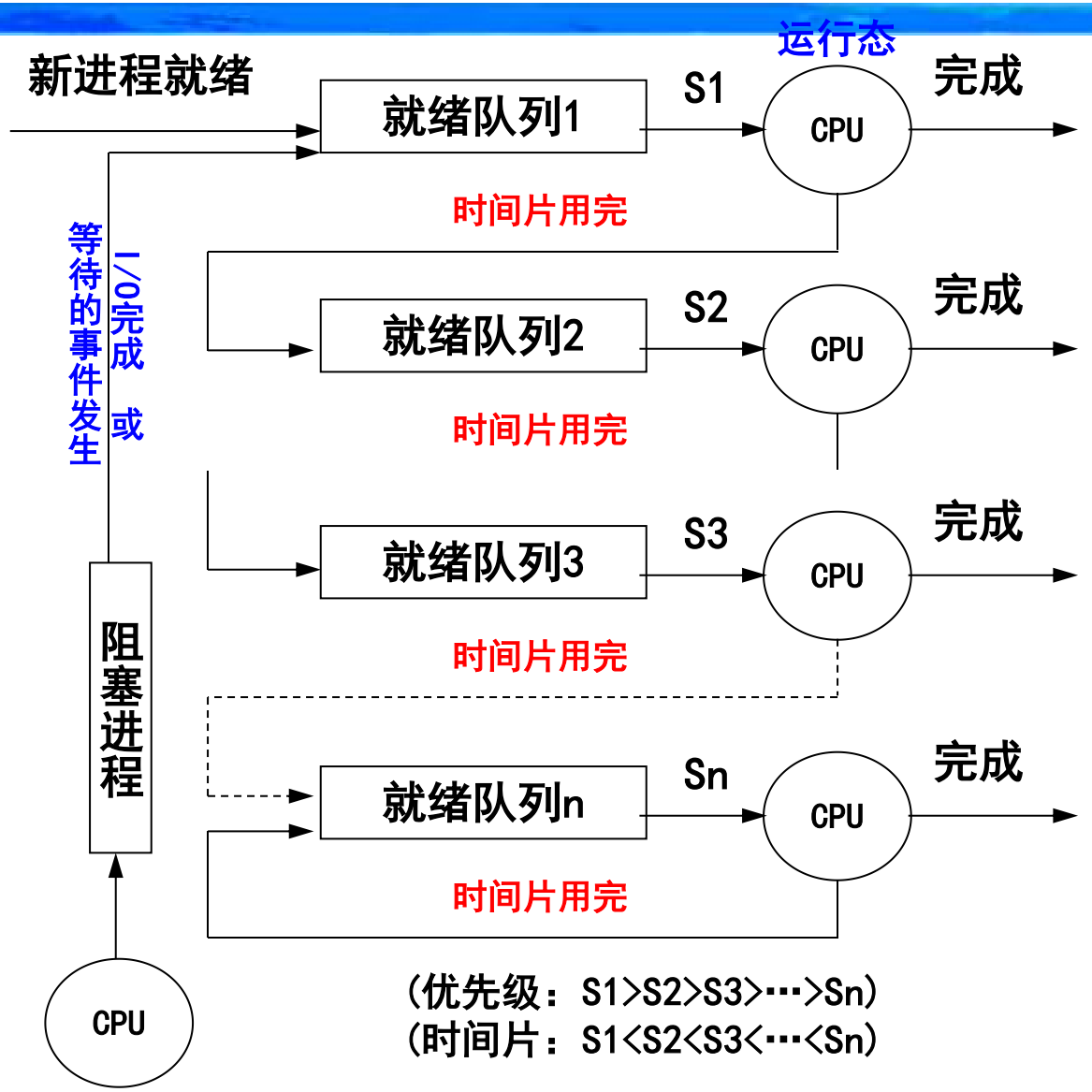


图4-3 多级反馈队列调度算法

4.3 实时调度

- **实现实时调度的基本条件**
  - **提供必要的信息**：就绪时间、开始截止时间和完成截止时间、处理时间、资源要求、优先级等。
  - **系统处理能力强**：假定系统中有m个周期性硬实时任务，它们的处理时间为C_i，周期时间为P_i，则在单处理机情况下，必须满足下面的限制条件，系统才是可调度的。

$$\sum_{i=1}^{m}\frac{C_i}{P_i}=\frac{C_1}{P_1}+\frac{C_2}{P_2}+\ldots+\frac{C_m}{P_m}\le 1$$

单处理机系统

$$\sum_{i=1}^{m}\frac{C_i}{P_i}\le N$$

多处理机系统

注意：上述限制条件并未考虑任务的切换时间，包括执行调度算法和进程任务切换，以及消息的传递时间等开销，因此，当利用上述限制条件来确定系统是否可调度时，还应适当地留有余地。

4.3 实时调度

实现实时调度的基本条件

- **采用抢占式调度机制**：
  - **硬实时任务**：抢占机制，满足硬实时任务对截止时间的要求
  - **小的实时系统**如果能预知任务的开始截止时间，则对实时任务的调度可采用非抢占调度机制，以简化调度程序和系统开销。
  - **实时任务适当小**：在执行完关键性程序和临界区后，能及时地把自己阻塞起来，以便释放处理机，供调度程序去调度那种开始截止时间即将到达的任务。
- **具有快速切换机制**
  - **对外部中断的快速响应能力**：要求系统具有快速硬件中断机构，还应使禁止中断的时间间隔尽量短，以免耽误其它紧迫任务。
  - **快速的任务切换能力**：使系统中的每个运行功能单位适当的小，以减少任务切换的时间开销。

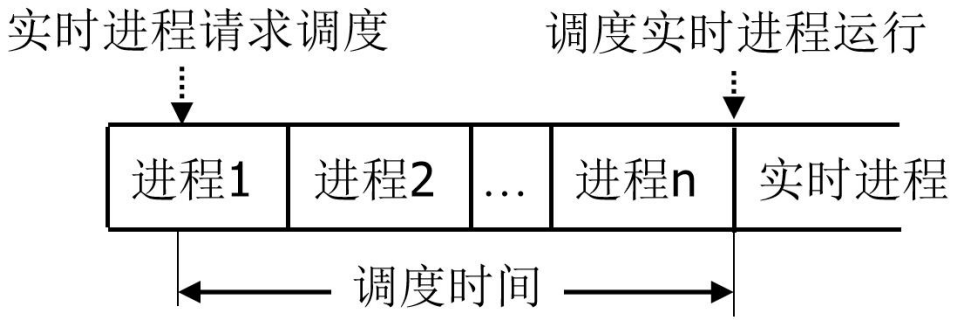
4.3 实时调度

实时调度算法分类

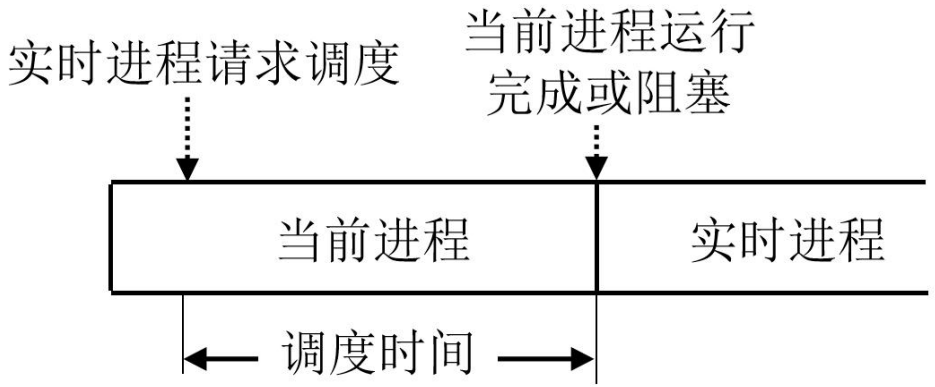
- 按实时任务性质不同来划分：
  - 硬实时调度算法：任务有严格的截止时间要求（硬实时任务），系统必须确保任务在截止时间之前完成。
  - 软实时调度算法：任务有相对灵活的截止时间要求（软实时任务），系统尽量确保任务在截止时间之前完成，系统有一定容错性。
- 按调度方式的不同
  - 非抢占调度算法
  - 抢占调度算法
- 按调度程序调度时间的不同
  - 静态调度算法：调度方案一旦确定，在运行过程中不会改变。
  - 动态调度算法：系统运行过程中动态决定调度方案。
- 多处理机环境下
  - 集中式调度
  - 分布式调度

4.3 实时调度

- **抢占/非抢占式实时调度算法**
  - **非抢占式实时调度算法**：算法比较简单，易于实现，故在一些小型实时系统或要求不太严格的实时控制系统中，经常采用。
    - **非抢占式轮转调度算法**

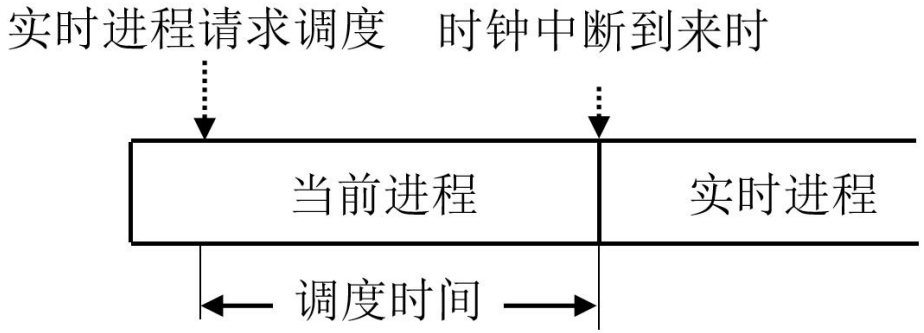


    - **非抢占式优先调度算法**

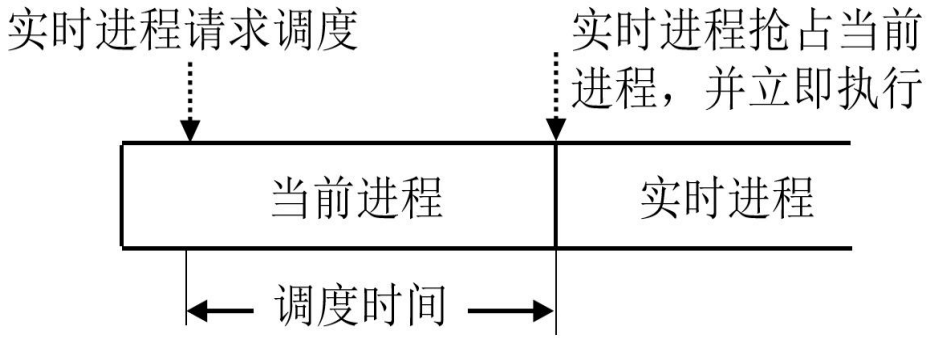


4.3 实时调度

- **抢占/非抢占式实时调度算法**
  - **抢占式实时调度算法：在要求较严格(响应时间为数十毫秒以下)的实时系统中采用。**
    - **基于时钟中断抢占的优先调度**



    - **立即抢占的优先权调度**



4.3 实时调度

- 硬实时调度算法
  - 最早截止时间优先算法(Earliest Deadling First，EDF）：根据任务的开始截止时间来确定任务的优先级。截止时间愈早，其优先级愈高，可用于抢占式调度，也可用于非抢占式调度。
  - 例：

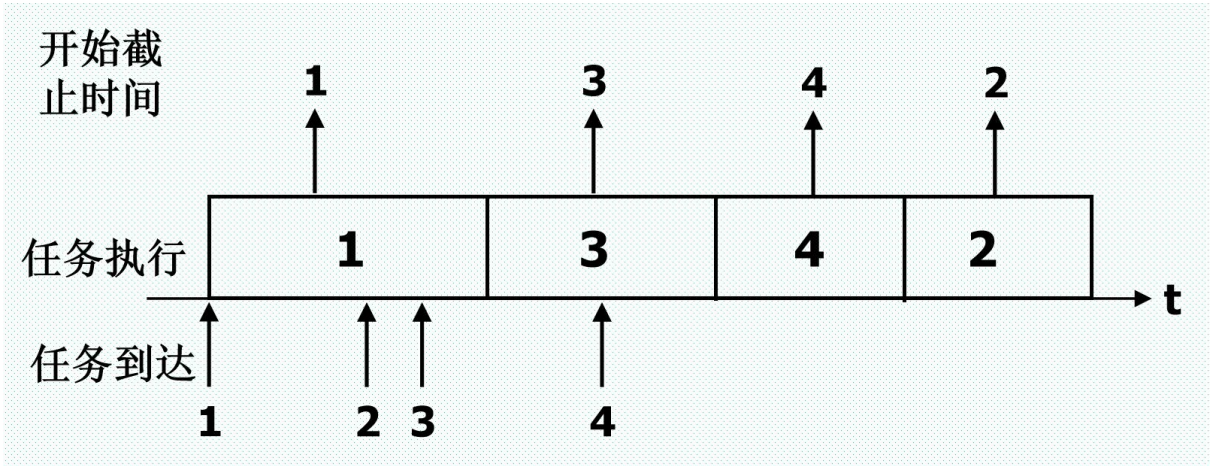


图4-4　EDF算法用于非抢占调度方式

4.5 实时调度

- 硬实时调度算法
  - 最早截止时间优先算法(Earliest Deadling First，EDF）：根据任务的开始截止时间来确定任务的优先级。截止时间愈早，其优先级愈高，可用于抢占式调度，也可用于非抢占式调度。
  - 例：

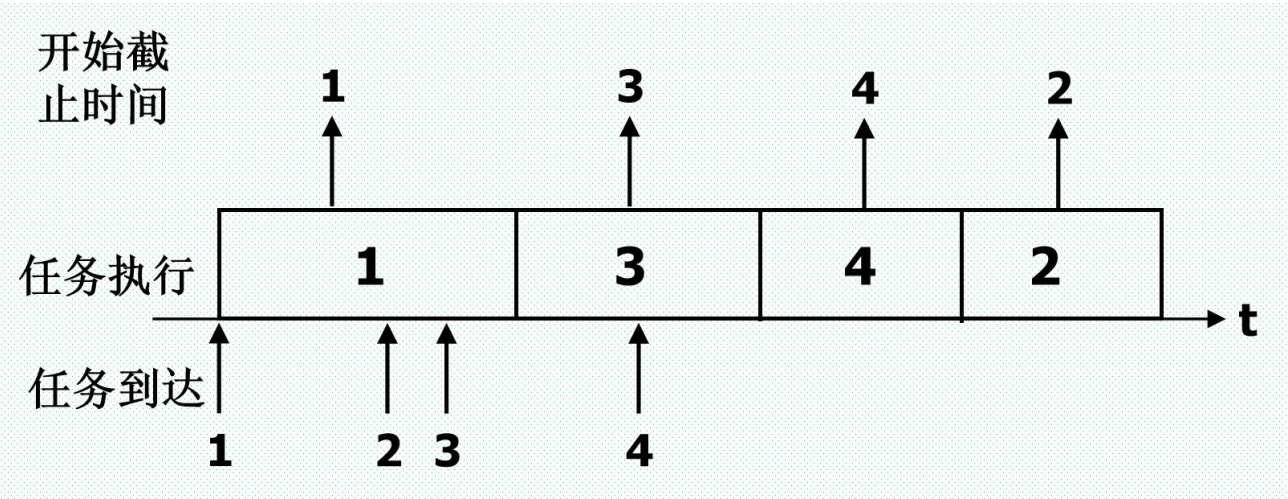


图4-4　EDF算法用于非抢占调度方式

4.5 实时调度

- 硬实时调度算法
  - 最低松弛度优先算法(Least Laxity First)，LLF）：根据任务紧急(或松弛)的程度来确定任务的优先级。任务紧急程度愈高，其优先级愈高。主要用于抢占式调度
  - 松弛度的计算：

    一个任务在200ms之前必须完成，它本身需要运行100ms，则该任务的紧急程度(松弛程度)为100ms。

例4：在一个实时系统中，有2个周期性实时任务A和B，任务A要求每20ms执行一次，执行时间为10ms；任务B要求每50ms执行一次，执行时间为25ms。由此可知任务A和任务B每次必须完成的时间分别为：A_1、A_2、A_3…和B_1、B_2、B_3…，见图4-4。为保证不遗漏任何一次截止时间，应采用最低松弛优先的抢占调度策略。

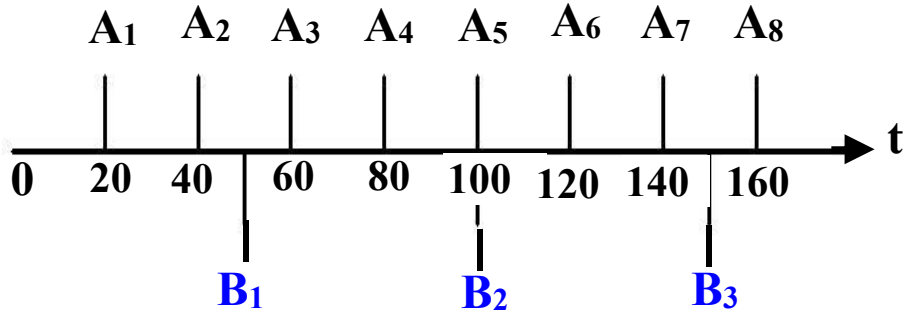
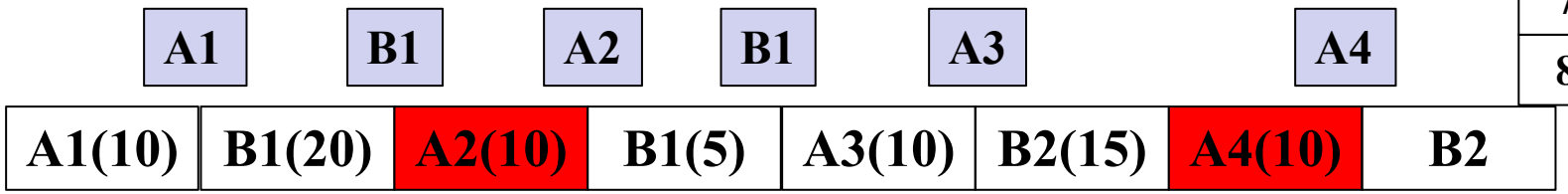
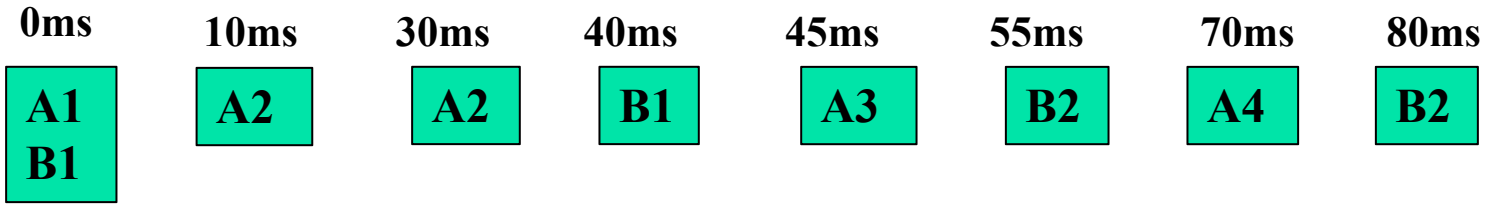


图4-4 A和B任务每次必须完成时间

t	$LL(A)_t$	$LL(B)_t$
0	10	25
10	20	15
30	0	15
40	10	5
45	5	30
55		
70	0	20
80	10	10





4.3 实时调度

详细过程

① 在刚开始时(t_1=0)，A_1必须在20ms时完成，而它本身运行需要10ms，可算出A_1的松弛度为10ms；B_1必须在50ms时完成，而它本身就需25ms，可算出B_1的松弛度为25ms，故调度程序应先调度A_1执行。

② 在t_2=10ms时，A_2的松弛度可按下式算出：

A_2的松弛度=必须完成的时间-其本身的运行时间-当前时间 =40ms-10ms-10ms=20ms

类似地，可算出B_1的松弛度为15ms，故调度程序应选择B_1执行。

③ 在t_3=30ms时，A_2的松弛度已减为0(即40-10-30)，而B_1的松弛度为15ms(50-5-30)，于是调度程序应抢占B_1的处理机而调度A_2运行。

④ 在t_4=40ms时，A_2完毕，A_3的松弛度为10ms(即60-10-40)，而B_1的松弛度为5ms(50-5-40)，故又重新调度B_1执行。

⑤ 在t_5=45ms时，B_1执行完毕，而此时A_3的松弛度已减为5(即60-10-45)，而B_2的松弛度为30ms(100-25-45)，于是又应调度A_3执行。

4.3 实时调度

详细过程

⑥ 在t_6=55ms时，A_3 执行完毕，任务A此时尚未进入A_4，而任务B已进入B_2周期，故调度B_2执行。

⑦ 在t_7=70ms时，A_4的松弛度已减为0(即80-10-70)，而B_2的松弛度为20ms(100-10-70)，于是调度程序应抢占B_2的处理机而调度A_4运行。

⑧ 在t_8=80ms时，A_4执行完毕。 A_5的松弛度为10ms(即100-10-80)，而B_2的松弛度为10ms(100-10-80)，故又重新调度B_2执行。……

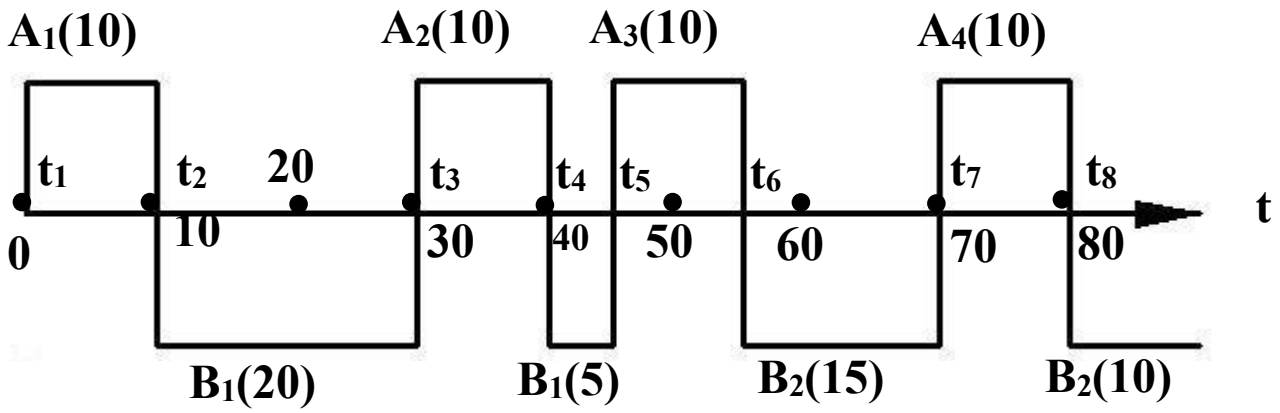


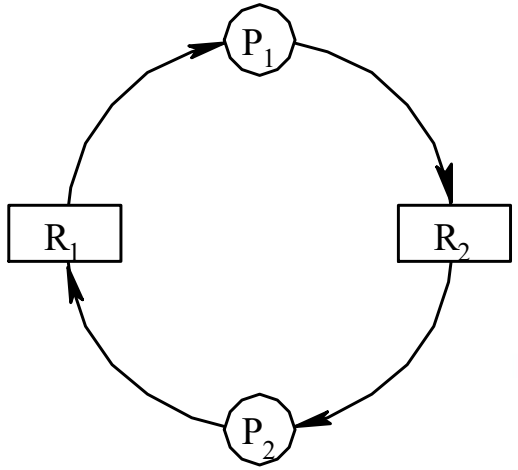
图4-5　利用LLF算法进行调度的情况

4.4 死锁

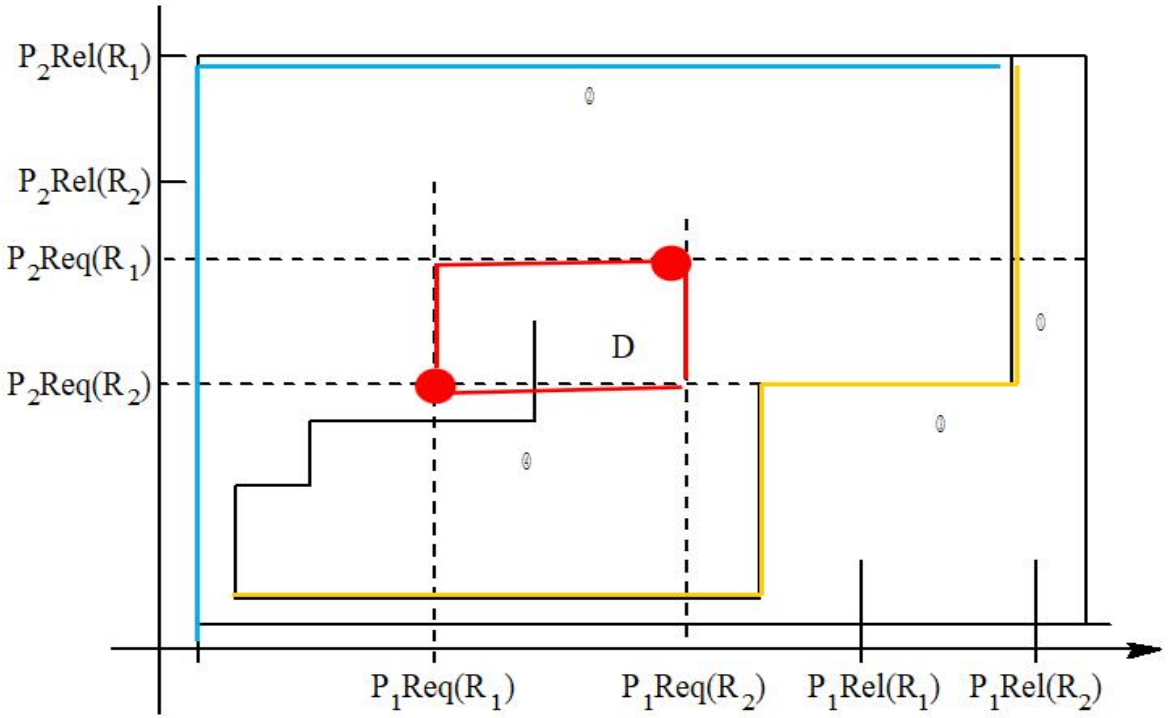
- 死锁（Deadlock）
  - 关于死锁的几个定义：
  - 多个进程在运行过程中因争夺资源而造成的一种僵局，当进程处于这种僵局状态时，若无外力作用，它们都将无法再向前推进。
  - 一组进程处于死锁状态是指：如果在一个进程集合中的每一个进程都在等待只能由该集合中的其它一个进程才能引发的事件，则称一组进程或系统发生了死锁。
  - 一组竞争系统资源或相互通信的进程间相互的“永久”阻塞。

4.4 死锁

- 产生死锁的原因
  - 竞争资源



  - 进程间推进顺序非法



4.4 死锁

- 产生死锁的必要条件
  - 互斥条件：进程对所分配到的资源进行排他性使用。
  - 请求和保持条件：进程提出了新的资源请求，但又对自己已获得的资源保持不放。
  - 不剥夺条件：进程已获得的资源，在未使用完之前，不能被剥夺。
  - 环路等待条件：发生死锁时，存在进程-资源的等待链。

4.4 死锁

处理死锁的基本方法

- **预防死锁**
  - 通过设置某些限制条件，去破坏产生死锁的四个必要条件中的一个或几个条件，来预防发生死锁。
  - 缺点：可能导致系统资源利用率和系统吞吐量的降低。
- **避免死锁**
  - 不需要事先采取各种限制措施去破坏产生死锁的四个必要条件，而是在资源动态分配过程中，用某种方法防止系统进入不安全状态，从而避免死锁。
- **检测死锁**
  - 允许系统在运行过程中发生死锁，但可通过系统设置的检测机构，及时检测出死锁的发生，然后采取适当的措施，从系统中将已发生的死锁清除掉。
- **解除死锁**
  - 与检测死锁相配套的措施。

4.4.1 预防死锁

- 破坏“互斥”条件
  - 如：Spooling技术，可以允许若干个进程同时产生输出。
  - 不足：Spooling技术并不适用于所有资源，而且它对磁盘空间的竞争也可能引起死锁。
- 摒弃“请求并保持”条件
  - 如：资源的一次性分配（静态分配）
  - 优点：简单，易实现。
  - 缺点：资源利用率低；使进程延迟运行（仅当能获得全部资源时，才能开始运行）

4.4.1 预防死锁

- 摒弃“不剥夺”条件
  - 当一个进程已经保持了某些资源，再提出新的资源请求而不能满足时，必须释放它已经保持了的资源，待以后需要时再重新申请。
  - 缺点：
    - 实现起来比较复杂且要付出很大代价。
    - 可能因为反复申请资源而使进程的执行被无限地推迟，增加了系统开销，降低了系统吞吐量。
- 摒弃“环路等待”条件
  - 采用资源“按号分配”
  - 缺点：
    - 进程使用资源的顺序与系统规定的顺序不同，造成资源浪费；
    - 会限制用户简单、自主地编程。

4.4.2 避免死锁

- 安全状态
  - 如果系统能按某种进程顺序P1，P2，…，Pn来为每个进程P分配其所需资源，确保满足每个进程对资源的最大需求，使每个进程都可顺利地完成，则称系统处于安全状态。<P1，P2，…，Pn>称为安全序列。
- 不安全状态
  - 定义：如果系统无法找到这样一个安全序列，则称系统处于不安全状态。
- 只要系统处于安全状态，便可避免进入死锁状态。

4.4.2 避免死锁

- 安全状态
  - 举例：设系统中有3个进程P1、P2、P3，共有12台磁带机。进程P1总共需要10台磁带机，P2和P3分别要求4台和9台。假设在T0时刻，进程P1、P2、P3已分别获得5、2、2台磁带机，尚有3台空闲未分配，如下表所示：

进程	最大需求	已分配	可用数
P1 P2 P3	10 4 9	5 2 2	3

  - 1）T0时刻系统是安全的，因为这时存在一个安全序列<P2，P1，P3>，即只要系统按此进程序列分配资源，就能使每一个进程都顺利完成。
  - 2）思考：在T_0时刻以后，P3又申请1台，系统将剩余的3台中的1台分配给P3，则系统便进入（　　）状态。　　不安全

4.4.2 避免死锁

- 银行家算法
  - 核心思想：在资源动态分配过程中，若分配后系统状态仍是安全的，则同意分配，否则将拒绝分配，这样可防止系统进入不安全状态，从而避免死锁。
  - 仍以前面为例：

进程	最大需求	已分配	可用数
P1	10	5	
P2	4	2	3
P3	9	2	

  - 思考：在T_0时刻以后，P3又申请1台，可以进行分配吗？

    不可以

4.4.2 避免死锁

- 银行家算法
  - 数据结构
    - 可利用资源向量Available：是一个含有m个元素的数组，其中每一个元素代表一类可用资源数目，m是资源种类数。
    - 最大需求矩阵Max：是一个n×m矩阵，它定义了系统中n个进程中的每一个进程对m类资源的最大需求数。
    - 分配矩阵Allocation：也是一个n×m矩阵，它定义了系统中每一类资源当前已分配给每一个进程的资源数。
    - 需求矩阵Need ：也是一个n×m矩阵，用于表示每个进程尚需的各类资源数。
  - 关系：Need [i,j] = Max[i,j]-Allocation[i,j]

4.4.2 避免死锁

- 银行家算法
  - 算法步骤：设$Request_i$是进程P_i的请求向量，如果$Request_i[j]=K$，表示进程P_i需要K个R_j类的资源。当P_i发出资源请求后，系统按下述步骤进行检查：
    - ①若$Request_i[j] \leqslant Need[i,j]$，转向步骤②；否则认为出错，因为它需要的资源数已超过它所宣布的最大值。
    - ②若$Request_i[j] \leqslant Available[j]$，转向步骤③；否则表示尚无足够资源，$P_i$须等待。
    - ③系统试探着把资源分配给进程P_i，并修改下面的数值：

      $Available[j] = Available[j] - Request_i[j]$

      $Allocation[i,j] = Allocation[i,j] + Request_i[j]$

      $Need[i,j] = Need[i,j] - Request_i[j]$
    - ④系统执行安全性算法，检查此次资源分配后，系统是否处于安全状态。若安全，才真正将资源分配给进程Pi，以完成本次分配；否则，将本次资源分配作废，恢复原来的资源分配状态，让进程P_i等待（阻塞）。

4.4.2 避免死锁

- 安全性检查算法
  - （1）设置两个向量。

    ①工作向量Work ：表示系统目前可提供的各类资源数，有m个元素。在执行本算法开始时，Work = Available。

    ②标志向量Finish：开始时Finish[i]=false （i=1,2,…,n)；当有足够资源分配给进程P_i时，再令Finish[i]=True；
  - （2）从进程集合中寻找一个能满足下述条件的进程：

    ①Finish[i]=False；②Need[i,j]≤Work[j]（j=1,2,…,m)

    若找到，转步骤（3）执行；否则，执行步骤（4）。
  - （3）进程P_i可获得所需全部资源，可执行结束并释放分配给它的资源。故应执行：

    Work[j] = Work[j] + Allocation[i, j]；Finish[i] = True；返回步骤（2）
  - （4）如果所有进程的Finish[i] == True都满足，则表示系统处于安全状态；否则，系统处于不安全状态。

4.4.2 避免死锁

- 银行家算法
  - 例2：设系统中有5个进程{P_0，P_1，P_2，P_3，P_4}和3类资源{A，B，C}，各类资源总数分别为10、5、7，在T_0时刻的资源分配情况如下表所示：

	Max			Allocation			Need			Available		
	A	B	C	A	B	C	A	B	C	A	B	C
P_0	7	5	3	0	1	0	7	4	3	3	3	2
P_1	3	2	2	2	0	0	1	2	2			
P_2	9	0	2	3	0	2	6	0	0			
P_3	2	2	2	2	1	1	0	1	1			
P_4	4	3	3	0	0	2	4	3	1			

  - T_0时刻系统是否安全，为什么？
  - P_1发出请求向量Request1（1，0，2），分析系统是否可同意请求？
  - 如果P_0发出请求向量$Request_4$（0，1，0），系统是否可同意请求？

4.4.2 避免死锁

- 例2：1）T_0时刻系统是安全的。利用安全性算法对T_0时刻的资源分配情况进行分析(见下表)可知，在T_0时刻存在着一个安全序列{P_1, P_3, P_4, P_2, P_0}，故系统是安全的。

资源情况 / 进程	Work A B C	Need A B C	Allocation A B C	Work+Allocation A B C	Finish
P_1	3 3 2	1 2 2	2 0 0	5 3 2	true
P_3	5 3 2	0 1 1	2 1 1	7 4 3	true
P_4	7 4 3	4 3 1	0 0 2	7 4 5	true
P_2	7 4 5	6 0 0	3 0 2	10 4 7	true
P_0	10 4 7	7 4 2	0 1 0	10 5 7	true

4.4.2 避免死锁

- 例2：P_1发出请求向量Request1（1，0，2），系统可同意。
  - 判断过程：
  - （1）$Request_1(1,0,2) \leq Need_1(1,2,2)$
  - （2）$Request_1(1,0,2) \leq Available(3,3,2)$
  - （3）系统先假定可为P_1分配资源，并修改Available，Allocation1和Need1向量，由此形成资源变化情况如下表所示：

	Max A B C	Allocation A B C	Need A B C	Available A B C
P_0	7 5 3	0 1 0	7 4 3	2 3 0
P_1	3 2 2	3 0 2	0 2 0	
P_2	9 0 2	3 0 2	6 0 0	
P_3	2 2 2	2 1 1	0 1 1	
P_4	4 3 3	0 0 2	4 3 1	

  - （4）再利用安全性算法检查此时系统是否安全。存在安全序列$\{P_1, P_3, P_4, P_2, P_0\}$，故系统是安全的，可以立即将$P_1$所申请的资源分配给它。

4.4.2 避免死锁

- 例2：P_4发出请求向量$Request_4(3,3,0)$，系统不能同意请求。
  - 判断过程：

    （1）$Request_4(3,3,0) \leqslant Need_4(4,3,1)$

    （2）$Request_4(3,3,0) \leqslant Available(2,3,0)$，让$P_4$等待

4.4.2 避免死锁

- 例2：P_0发出请求向量$Request_4$（0，2，0），按银行家算法，系统不能同意请求
  - 判断过程：

（1）$Request_0(0,2,0) \leq Need_0(7,4,3$

（2）$Request_0(3,3,0) \leq Available(2,3,0)$

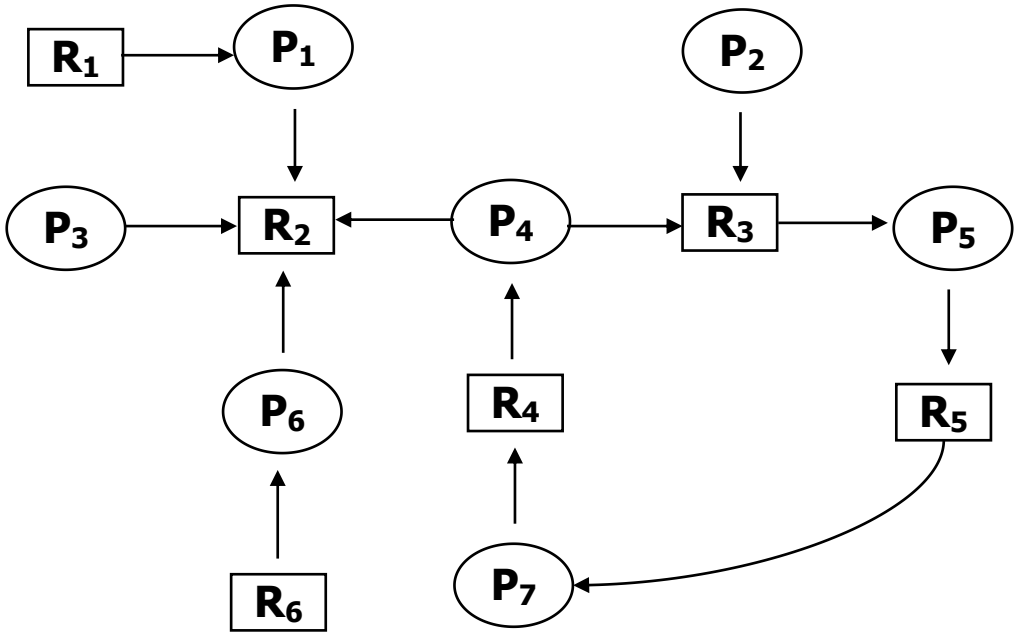
（3）系统暂时先假定可为P_0分配资源，修改有关数据，如下：

资源情况 / 进程	Max A B C	Allocation A B C	Need A B C	Available A B C
P_0	7 5 3	0 3 0	7 2 3	2 1 0
P_1	3 2 2	3 0 2	0 2 0	
P_2	9 0 2	3 0 2	6 0 0	
P_3	2 2 2	2 1 1	0 1 1	
P_4	4 3 3	0 0 2	4 3 1	

（4）进行安全性检查，可用资源Available(2,1,0)已不能满足任何进程的需要，系统进入不安全状态，故系统不能同意P0的请求，让其阻塞。

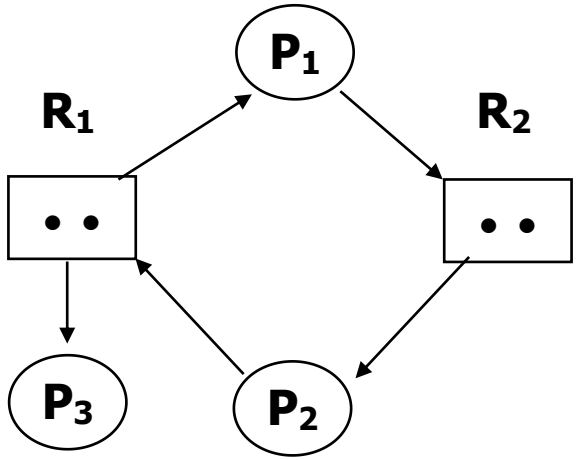
4.4.3 死锁检测

- 利用资源分配图检测死锁
  - 资源分配图



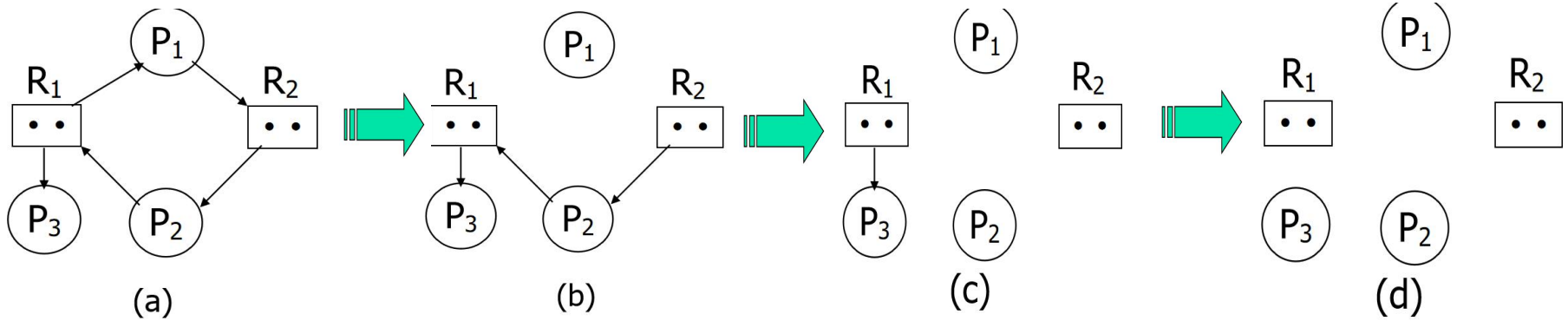
4.4.3 死锁检测

- 利用资源分配图检测死锁
- 判断方法
  - （1）若资源分配图中无环路，则此时系统中无死锁。
  - （2）如果资源分配图中有环路，且每个资源类中仅有一个资源，则系统中发生了死锁。此时，环路中的进程便为死锁进程。
  - （3）如果资源分配图中有环路，且涉及的资源类中有多个资源，则环路的存在只是产生死锁的必要条件而不是充要条件，系统未必一定会发生死锁。



4.4.3 死锁检测

- 利用资源分配图检测死锁
  - 对于第三种情况：资源分配图中有环路，且涉及的资源类中有多个资源的情况，可利用把资源分配图简化的方法，来检测系统是否存在死锁。



    - 经过一系列简化后，若能消去图中所有的边，使所有进程结点都成为孤立结点，则称该图是可完全简化的；否则称该图为不可完全简化的。
  - 死锁定理：系统为死锁状态的充要条件是：当且仅当该状态的资源分配图是不可完全简化的。

4.4.3 死锁检测

- **利用资源分配图检测死锁**
  - 对应的死锁检测算法：
  - 死锁检测中用到的数据结构包括：Available、Allocation、Request(请求矩阵)。检测步骤如下：
    ① 标记Allocation矩阵中一行全0的进程；
    ② 令临时向量Work=Available；
    ③ 查找下标i，进程P_i满足：尚未标记且Request的第i行小于等于Work。如果找不到这样的行，则终止查找
    ④ 若找到这样的行，则标记进程P_i，并把分配矩阵的相应行加到Work中。返回(3)
    - **当且仅当算法的最后结果有未标记的进程时，系统存在死锁，每个未标记的进程都是死锁的。**

4.4.4 死锁解除

- 剥夺资源
  - 如：
    - 从其它进程剥夺足够资源给死锁进程，以解除死锁状态。
    - 剥夺死锁进程占用的资源，但并不撤消它，直到死锁解除。
- 撤消进程
  - 如：
    - 撤消全部死锁进程
    - 按某个顺序逐个撤消死锁进程，直至使死锁状态解除为止。
      考量因素：进程优先级、已运行时间、已获得资源等。

4.4 死锁

- **练习：**
  - （1）哲学家就餐问题中解决死锁有哪几种方法，各是基于什么策略？
  - （2）某计算机系统有9台磁带机，它们供N个进程竞争使用，每个进程可能需要3台磁带机。请问N为多少时，系统没有死锁的危险，并说明其原因。
  - （3）某系统有同类资源m个供n个进程共享，如果每个进程最多可能需要x个资源(1≤x≤m)且各进程的最大需求量之和小于(m + n)。证明系统没有因申请该类资源而发生死锁的危险。
- **解答：**
  - （2）4
  - （3）证明：若系统会死锁，则n(x-1)+1≤m一定不成立，即n(x-1)+1＞m成立。于是nx＞m+n-1，推得nx≥m+n，与题意nx＜m+n矛盾，故系统不会死锁。